FUCAPE
BUSINESS SCHOOL

Busca Exata e Aproximada (Tolerant Retrieval)

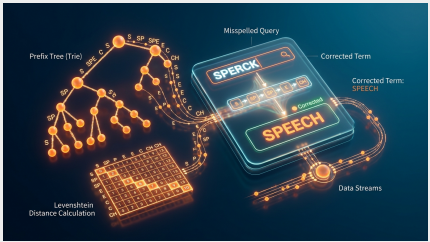
PLN e Sistemas de Recuperação de Informação

Prof. Me. Wagner Perin
FUCAPE Business School

Busca Exata e Aproximada (Tolerant Retrieval)

Árvores Trie, Índices de k-gramas, Distância de Levenshtein e Fonética Soundex

Disciplina: PLN e Sistemas de Recuperação de Informação
Programa: Ciência de Dados para Negócios — FUCAPE Business School
Professor: Prof. Me. Wagner Perin
Carga Horária: 4 h/a (200 minutos) | **Unidade 3:** Processamento e Resolução de Consultas



INSIGHT EXECUTIVO

Escopo Arquitetural da Aula 07:
Superar a fragilidade do casamento léxico exato implementando estruturas industriais de recuperação tolerante a falhas: **Árvores de Prefixo (Tries)** em $O(|P|)$, **Índices Secundários de k-gramas ($)** para curingas, pipeline em 2 estágios (**Jaccard + Programação Dinâmica de Levenshtein**) e **Hashing Fonético Soundex**.

Resgate & O Ponto Cego do Casamento Exato

Da Velocidade Sublinear do WAND à Fragilidade diante de Ruído Ortográfico

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

A Premissa Silenciosa e Frágil: Toda a maquinaria algorítmica construída até a Aula Anterior assume que o termo digitado pelo usuário existe exatamente igual no Dicionário do Índice Invertido.

O Que Conquistamos na Aula Anterior vs. A Lacuna do Mundo Real:

- **Conquistas da Aula Anterior:**
  1. Avaliador booleano com **Shunting-Yard (RPN)** e interseção ordenada pela menor frequência documental (DF_t);
  2. **Índices Posicionais** ($\langle$DocID, tf, [pos]$\rangle$) para frases exatas (`"dano moral"`) e proximidade (`/k`);
  3. Ranking escalável Top-K via **Min-Heap** ($O(N \log K)$), **Tiered Indexes** e poda dinâmica segura **WAND** (UB$_t$).
- **O Ponto Cego Crítico (`dictionary.get(term) -> None`):**
  - Todo algoritmo da Aula Anterior começa consultando a chave do termo no vocabulário em RAM.
  - Se o cliente digita `"samsumg"`, `"notebuk"`, o radical `"ba*co"` ou o sobrenome `"Sousa"` (grafado `"Souza"` no contrato), `dictionary.get(term)` retorna `None` e o motor mais rápido do mundo devolve `0 resultados` **em** $0,4$ **ms**!

A Dor de Negócio I — *Zero-Result Searches*

O Impacto Financeiro da Digitação Mobile no GMV do E-commerce

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Problema Investigado: No MegaVarejo S.A., 18,4% de todas as buscas no aplicativo móvel retornam página vazia por erros de digitação, gerando abandono imediato de sessão.



Radiografia do Relatório de *Search Analytics*:

- **Realidade Mobile: 72% do tráfego** vem de smartphones, onde a digitação com o polegar em movimento (*fat-finger syndrome*) multiplica erros tipográficos.
- **Top-3 Consultas Abandonadas (*Zero-Result Searches*):**
  - `"samsumg"` (`24.500` buscas — troca de `'n'` por `'m'`);
  - `"notebuk"` (`18.200` buscas — troca de `'o'` por `'u'` e omissão de `'o'`);
  - `"ipone 15"` (`11.900` buscas — omissão da letra `'h'`).
- **Impacto Financeiro Direto:** Perda estimada de **R$ 4,2 milhões em GMV** (*Gross Merchandise Value*) em um único fim de semana — ou **R$ 350.000/mês** recorrentes em nichos específicos!

A Dor de Negócio II — Colapso da Força-Bruta

Por que Varreduras Lineares $O(|V|)$ Destroem o SLA em Produção

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

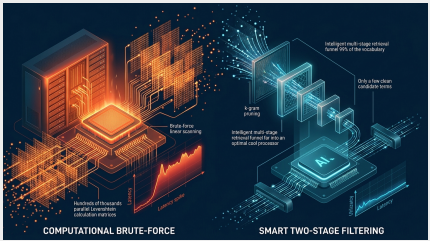
Desastre em Produção: Na tentativa de consertar os erros de digitação e buscas com curinga, um desenvolvedor varreu linearmente o vocabulário de $|V| = 440.000$ termos a cada consulta, derrubando o cluster.

Por que a Força-Bruta e o `LIKE '%...%'` Destroem o Servidor?

1. **Consultas com Curinga (`ba*co`, `comp*`) via Scan Linear:** Rodar `re.match` ou `SQL LIKE '%ba%co%'` exige inspecionar todas as 440.000 chaves a cada requisição ($O(|V|)$).
2. **A Conta Explosiva do Levenshtein Força-Bruta:** Calcular uma matriz DP entre $m = 8$ e $n = 8$ letras exige $8 \times 8 = 64$ células. Contra todo o vocabulário $|V| = 440.000$:

Custo/Busca $= |V| \cdot m \cdot n = 440.000 \times 64 = \mathbf{28,16 \times 10^6}$ células!

- Sob 1.000 QPS, são $28,16$ **bilhões de operações/s**, derrubando a vazão para **2 QPS** e gerando *timeout* generalizado!

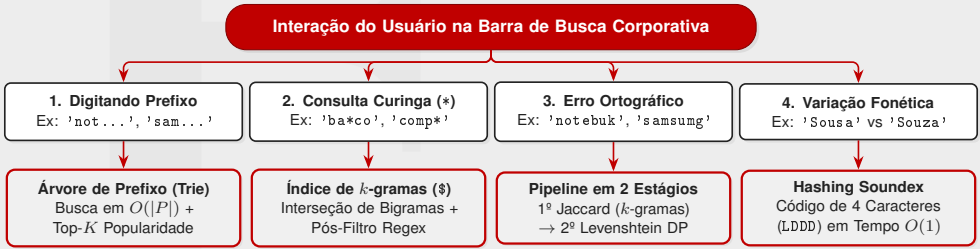


A Jornada da Tolerância — 4 Problemas, 4 Estruturas

Mapeamento Arquitetural de Falhas Léxicas para Índices Secundários

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Estratégia de Engenharia (Diluição de Carga Cognitiva): Cada falha léxica ocorre em um momento distinto da interação do usuário e exige uma estrutura de dados especializada.



INSIGHT EXECUTIVO

Princípio Fundamental de RI: Nunca executamos algoritmos pesados (Regex ou Programação Dinâmica) sobre o vocabulário inteiro $|V|$; usamos **estruturas de acesso e índices secundários sobre o vocabulário** para podar $99,9\%$ do espaço!

Acesso ao Vocabulário — Hash vs. B-Tree vs. Trie

Complexidade Assintótica por Modalidade de Consulta no Léxico

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema de Arquitetura & Decisão Analítica: Por que uma Tabela Hash tradicional — que faz busca exata em $O(1)$ — é totalmente inútil quando o usuário digita apenas o prefixo `'note'` no auto-complete?

Comparativo Arquitetural de Dicionários em Memória ($|V|$ palavras, prefixo de tamanho $|P|$):

Operação de Busca no Vocabulário	Tabela Hash (*Hash Map*)	Árvore B (*B-Tree* / Array)	Árvore de Prefixo (*Trie*)	Índice de k-gramas ($)
1. Busca Exata (`"notebook"`)	$O(1)$ **médio**	$O(\log_2 \|V\|)$	$O(\|P\|)$	Não aplicável (auxiliar)
2. Prefixo / Auto-complete (`not*`)	$O(\|V\|)$ *(Colapso!)*	$O(\log_2 \|V\| + R)$	$O(\|P\|)$ ***(Ótimo!)***	$O(L_{\text{postings}})$
3. Sufixo (`*book`)	$O(\|V\|)$ *(Inviável)*	$O(\|V\|)$ *(Exige Trie Reversa)*	$O(\|V\|)$ *(Exige Trie Reversa)*	**Suporte Nativo Rápido**
4. Curinga Interno (`ba*co`)	$O(\|V\|)$ *(Inviável)*	$O(\|V\|)$ *(Inviável)*	$O(\|V\|)$ *(Branching alto)*	**Suporte Nativo Rápido**

- **A Falha da Hash:** A função `hash("not")` e `hash("notebook")` geram endereços de memória completamente desconexos, destruindo a vizinhança lexicográfica!

Árvores de Prefixo (*Tries*) para Auto-complete

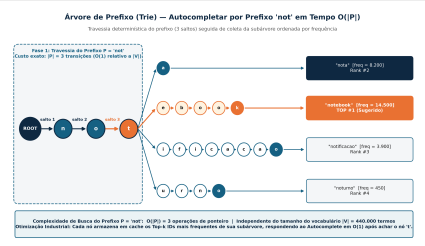
Navegação Determinística em $O(|P|)$ Independente da Escala do Vocabulário

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Alinhamento Direto (Lista de Exercícios — Item 3.10): Como sugerir continuações de digitação em menos de 2 ms ordenadas pela popularidade histórica de busca?

Anatomia e Complexidade da *Prefix Trie*:

1. **Estrutura de Grafo em Árvore:**
   - Cada **aresta** representa um caractere (`'n'`, `'o'`, `'t'`).
   - Cada **nó terminal** armazena `is_end_of_word` e a popularidade (`frequency`) no log de buscas.
2. **Invariância ao Tamanho do Vocabulário ($O(|P|)$):**
   - Para localizar o nó do prefixo `"not"` ($|P| = 3$), o motor realiza **exatamente 3 saltos de ponteiro**, quer o vocabulário tenha 10.000 ou 100 milhões de palavras!
3. **Recuperação Top-K de Sugestões:**
   - A partir do nó `"not"`, percorre-se a subárvore (ou lista Top-K em cache no nó) retornando `[("notebook", 14500), ("nota", 8200), ("notificacao", 3900)]`.



Implementação em Python — PrefixTrie

Micro-Etapa 1 do Desafio PBL: Sugestão Instantânea por Prefixo em $O(|P|)$

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Micro-Etapa 1 do Desafio Prático (PBL): Estrutura de dados enxuta em Python puro para sugestão instantânea de consultas por prefixo.

```python
class TrieNode:
    """Nó da Árvore de Prefixo (Trie) armazenando transições e popularidade."""
    def __init__(self) -> None:
        self.children: dict[str, TrieNode] = {}
        self.is_end_of_word: bool = False
        self.term: str = ""
        self.frequency: int = 0
class PrefixTrie:
    def __init__(self) -> None:
        self.root = TrieNode()
    def insert(self, term: str, frequency: int = 1) -> None:
        node = self.root
        for char in term.lower():
            node = node.children.setdefault(char, TrieNode())
        node.is_end_of_word, node.term, node.frequency = True, term.lower(), frequency
    def autocomplete(self, prefix: str, top_k: int = 5) -> list[tuple[str, int]]:
        node = self.root
        for char in prefix.lower():             # Complexidade O(|P|) saltos!
            if char not in node.children: return []
            node = node.children[char]
        results: list[tuple[str, int]] = []
        self._dfs_collect(node, results)        # Coleta palavras da subárvore
        return sorted(results, key=lambda x: (-x[1], x[0]))[:top_k]
```

O Desafio das Consultas com Curinga (*)

Limitações de Tries em Sufixos/Meio de Palavra e o Custo do Permuterm Index

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema de Arquitetura: Quando o analista fiscal ou jurídico pesquisa `ba*co`, `pro*o` *ou* `*cao`, *por que uma Árvore Trie padrão entra em colapso?*

As 3 Modalidades de Curinga (*) e seus Gargalos:

1. **Curinga no Final (*Trailing Wildcard* — `comp*`):**
   - Resolvido diretamente pela *Trie* ou Árvore B em $O(|P|)$.
2. **Curinga no Início (*Leading Wildcard* — `*book` ou `*cao`):**
   - Na *Trie* normal, não sabemos por qual ramo da raiz começar! Exigiria manter uma **Segunda Trie Invertida (*Reverse Trie*)** lendo as palavras de trás para frente (`"koob*"`).
3. **Curinga no Meio ou Múltiplos Curingas (`ba*co`, `pro*o`, `*mp*`):**
   - Mesmo cruzando uma *Trie* direta (`ba*`) com uma *Trie* reversa (`*co` $\rightarrow$ `oc*`), a interseção de conjuntos enormes de prefixos e sufixos é cara, e falha para múltiplos curingas (`c*mp*cao`).
   - **E o *Permuterm Index*?** Rotacionar todas as posições de `$banco` (`banco$`, `anco$b`, `nco$ba`...) multiplica o tamanho do dicionário em $6\times$ **a** $8\times$ **na RAM**! Precisamos de uma solução mais compacta.

Índice Secundário de k-gramas e Marcador $

Indexando Fragmentos do Próprio Vocabulário com Marcadores de Fronteira

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Alinhamento Direto (Lista de Exercícios Item 3.8 e Prova A1 Q14): Construindo um "índice invertido em miniatura" onde os documentos são as palavras do vocabulário e os termos são fragmentos de k letras!

Como Funciona a Decomposição em k-gramas ($k = 2$ Bigramas):

1. **Adição dos Marcadores de Fronteira ($):** Para ancorar exatamente onde a palavra **começa** e onde ela **termina**, envolvemos cada termo do vocabulário com o símbolo especial $: palavra $\longrightarrow$ `$palavra$`.
2. **Janela Deslizante de Tamanho k:** Para uma palavra de comprimento L, a string emoldurada tem tamanho $L + 2$, gerando exatamente $L + 3 - k$ fragmentos (para bigramas $k = 2$, são $L + 1$ **bigramas**).

INSIGHT EXECUTIVO – Exemplos Clássicos de Decomposição (Prova A1 & Lista de Exercícios)

- **Caso 1 (Prova A1 — Q14):** Termo `"banco"` ($L = 5$) $\longrightarrow$ String emoldurada: `"$banco$"` $\Longrightarrow$ $\{$`"$b"`, `"ba"`, `"an"`, `"nc"`, `"co"`, `"o$"`$\}$ (6 bigramas).
- **Caso 2 (Lista — Item 3.8):** Termo `"projeto"` ($L = 7$) $\longrightarrow$ String emoldurada: `"$projeto$"` $\Longrightarrow$ $\{$`"$p"`, `"pr"`, `"ro"`, `"oj"`, `"je"`, `"et"`, `"to"`, `"o$"`$\}$ (8 bigramas).

Rastreamento — Consulta Curinga `ba*co`

Decomposição em Bigramas e Interseção Booleana das Listas do Vocabulário

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

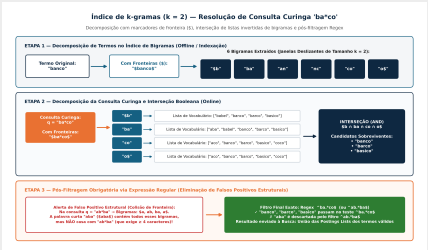
Rastreamento Passo a Passo (Prova A1 — Q14): Como o motor resolve `ba*co` em microssegundos sem varrer o dicionário?

Passo 1: Decomposição da Consulta Curinga

- Emolduramos a busca com $: `"$ba*co$"`.
- Extraímos os bigramas dos trechos conhecidos (ignorando o *):
  Consulta k-gramas = `"$b" AND "ba" AND "co" AND "o$"`

Passo 2: Interseção das Listas do Índice de k-gramas

- `"$b"` → [“bala”, “banco”, “barco”, “barranco”, “basco”]
- `"ba"` → [“abacate”, “bala”, “banco”, “barco”, “barranco”, “basco”]
- `"co"` → [“banco”, “barco”, “barranco”, “basco”, “coco”, “foco”]
- `"o$"` → [“banco”, “barco”, “barranco”, “basco”, “coco”, “foco”]
- **Interseção (`AND`):** Sobrevivem `["banco", "barco", "barranco", "basco"]`!



Falsos Positivos e *Post-filtering* Obrigatório

Por que a Interseção de k-gramas Exige Verificação Final por Regex

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Pegadinha Clássica de Engenharia e de Prova: A interseção das listas do índice de k-gramas é suficiente para garantir que a palavra candidata casa com a consulta curinga?

Por que a Interseção de k-gramas Pode Gerar Falsos Positivos?

- O índice de k-gramas verifica apenas se os fragmentos **existem** dentro da palavra, mas **perde a ordem global e a distância mínima** entre eles!
- **Exemplo Concreto de Falso Positivo (`ab*ba`):**
  - Consulta: `ab*ba` $\longrightarrow$ Emoldurada: `"$ab*ba$"` $\longrightarrow$ Exige palavras com pelo menos 4 letras (`^ab.*ba$`).
  - Bigramas buscados: `"$a" AND "ab" AND "ba" AND "a$"`.
  - Considere a palavra curta `"aba"` ($L = 3$, emoldurada `"$aba$"`): seus bigramas são exatamente `{"$a", "ab", "ba", "a$"}`!
  - Logo, `"aba"` **passa na interseção booleana**, mas **NÃO casa com** `ab*ba` (pois o `'b'` foi contado em sobreposição dupla)!

INSIGHT EXECUTIVO – A Regra de Ouro: Post-filtering via Expressão Regular

Após a interseção rápida do índice de k-gramas (que reduz 440.000 termos para meia dúzia de candidatos), o motor aplica um **filtro final de Regex (`^ba.*co$` ou `^ab.*ba$`)** apenas sobre os sobreviventes antes de consultar o Índice Invertido principal!

Implementação em Python — Índice de k-gramas

Micro-Etapa 2 do Desafio PBL: Decomposição com $ e Filtro Regex de Precisão

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

*Micro-Etapa 2 do Desafio Prático (PBL): Extração de bigramas com marcador **$** e resolução de curingas com pós-filtragem.*

```python
class KGramVocabularyIndex:
    def __init__(self, k: int = 2) -> None:
        self.k = k
        self.kgram_index: dict[str, set[str]] = defaultdict(set)
        self.vocab_kgrams: dict[str, set[str]] = {}
    def extract_kgrams(self, word: str, add_boundaries: bool = True) -> list[str]:
        """Ex (k=2): 'banco' -> '$banco$' -> ['$b', 'ba', 'an', 'nc', 'co', 'o$']"""
        w = f"${word.lower()}$" if add_boundaries else word.lower()
        return [w[i:i + self.k] for i in range(len(w) - self.k + 1)] if len(w) >= self.k else [w]
    def wildcard_search(self, pattern: str) -> list[str]:
        framed = f"${pattern.lower()}$"
        query_kgrams = [seg[i:i+self.k] for seg in framed.split("*")
                        if len(seg) >= self.k for i in range(len(seg) - self.k + 1)]
        # Passo 1: Interseção booleana ordenada pela menor lista de k-gramas
        sets = sorted([self.kgram_index.get(kg, set()) for kg in query_kgrams], key=len)
        raw_candidates = set.intersection(*sets) if sets else set(self.vocab_kgrams.keys())
        # Passo 2: Post-filtering obrigatório via Regex apenas nos candidatos sobreviventes!
        regex = re.compile("^" + ".*".join(re.escape(p) for p in pattern.lower().split("*")) + "$")
        return sorted(cand for cand in raw_candidates if regex.match(cand))
```

Correção Ortográfica — Precisão vs. Latência

Critérios Formais para Eleição de Candidatos em Buscas com Erro Tipográfico

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Desafio de Engenharia: Como corrigir automaticamente "samsumg" → "samsung" e sugerir "notebuk" → "notebook" sem jamais rodar Programação Dinâmica contra todo o vocabulário?

As Duas Formas de Correção Ortográfica em RI:

1. **Correção de Termo Isolado (*Isolated-Term Correction* — Foco desta Aula):**
   - Verifica cada token da busca individualmente. Acionada quando o termo **não existe no vocabulário** ($DF_t = 0$, ex.: `"notebuk"`) ou possui frequência documental extremamente baixa ($DF_t \approx 0$).
2. **Correção Sensível ao Contexto (*Context-Sensitive Correction*):**
   - Quando os termos existem isoladamente no dicionário, mas a combinação tem zero resultados (ex.: `"voo de aviao"` grafado como `"vou de aviao"`, onde `"vou"` é verbo válido!).

INSIGHT EXECUTIVO – O Princípio de Seleção do Melhor Candidato (C^*)

Combinamos **Proximidade Gráfica** (menor Distância de Edição $D(Q, C)$) com **Prior de Popularidade no Catálogo** (maior frequência DF_C ou frequência no log de buscas):

$$\text{Melhor Candidato } C^* = \arg \min_{C \in \mathcal{V}_{\text{filtrado}}} D_{\text{Levenshtein}}(Q, C) \quad \text{(desempatado pelo maior } DF_C\text{)}$$

Arquitetura em 2 Estágios — Jaccard + Levenshtein

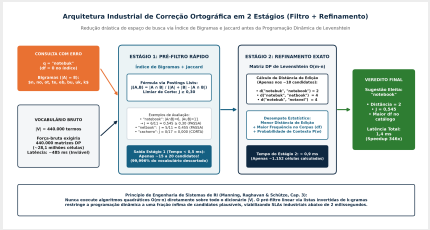
Reaproveitando o Índice de k-gramas para Poda de 99,996% do Vocabulário

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

A Sacada Arquitetural que Salva a CPU: Reaproveitamos o mesmo Índice de k-gramas de Caracteres para filtrar o vocabulário de 440.000 para ≈ 15 palavras antes de rodar Levenshtein!

O Funil de Baixa Latência (< 1 ms total):

- **Entrada:** Usuário digita `"notebuk"` ($DF = 0$).
- **Estágio 1 — Geração Rápida de Candidatos (*Candidate Generation*):**
  - Quebramos `"notebuk"` em bigramas e consultamos o Índice de k-gramas.
  - Calculamos o **Coeficiente de Jaccard** $J(G_Q, G_V)$ apenas para palavras que compartilham bigramas e retemos aquelas com $J \geq \tau$ (ex.: $\tau = 0,25$).
  - **Poda:** De 440.000 palavras, sobram apenas ≈ 15 **candidatas**!
- **Estágio 2 — Ranqueamento Exato (*Exact Edit Distance*):**
  - Calculamos a matriz de **Levenshtein DP** $(m+1) \times (n+1)$ **somente para as 15 sobreviventes**!



Estágio 1 — Coeficiente de Jaccard em k-gramas

Pré-Filtragem Rápida por Sobreposição de Bigramas de Caracteres

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Por que o Coeficiente de Jaccard pode ser calculado em sub-milissegundo diretamente sobre as listas invertidas do Índice de k-gramas?

Formulação Matemática da Sobreposição de k-gramas:

Dados o conjunto de bigramas da consulta G_Q e o conjunto de bigramas de uma palavra do vocabulário G_V:

$$J(G_Q, G_V) = \frac{|G_Q \cap G_V|}{|G_Q \cup G_V|} = \frac{|G_Q \cap G_V|}{|G_Q| + |G_V| - |G_Q \cap G_V|}$$

Exemplo Numérico Passo a Passo (`Q = "notebuk"`):

- Bigramas de `"$notebuk$"`: $G_Q = \{\texttt{"\$n"}, \texttt{"no"}, \texttt{"ot"}, \texttt{"te"}, \texttt{"eb"}, \texttt{"bu"}, \texttt{"uk"}, \texttt{"k\$"}\} \implies |G_Q| = 8$.
- **1. Candidato** $V_1 = \texttt{"notebook"}$ ($G_{V_1} = \{\texttt{"\$n"}, \texttt{"no"}, \texttt{"ot"}, \texttt{"te"}, \texttt{"eb"}, \texttt{"bo"}, \texttt{"oo"}, \texttt{"ok"}, \texttt{"k\$"}\} \implies |G_{V_1}| = 9$):
  - Interseção $G_Q \cap G_{V_1} = \{\texttt{"\$n"}, \texttt{"no"}, \texttt{"ot"}, \texttt{"te"}, \texttt{"eb"}, \texttt{"k\$"}\} \implies \mathbf{|G_Q \cap G_{V_1}| = 6}$.
  - União $|G_Q \cup G_{V_1}| = 8 + 9 - 6 = 11 \implies \mathbf{J}(\text{“notebuk”, “notebook”}) = \frac{6}{11} \approx \mathbf{0,545}$ (**Aprovado!** $\geq 0,25$).
- **2. Candidato** $V_2 = \texttt{"noturno"}$ ($|G_{V_2}| = 8$, compartilha apenas `{"$n", "no", "ot"}` $\implies |G_Q \cap G_{V_2}| = 3$):
  - União $= 8 + 8 - 3 = 13 \implies \mathbf{J}(\text{“notebuk”, “noturno”}) = \frac{3}{13} \approx \mathbf{0,231}$ (**Descartado no pré-filtro!**).

Estágio 2 — Distância de Edição de Levenshtein

Métrica Formal de Transformação de Strings por Inserção, Deleção e Substituição

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Alinhamento Direto (Lista de Exercícios Item 3.7 e Prova A1 Q13): O que mede formalmente a Distância de Levenshtein entre duas strings s_1 e s_2?

As 3 Operações Atômicas de Edição (Custo Unitário Padrão $= 1$):

A **Distância de Edição de Levenshtein** $D(s_1, s_2)$ é o **número mínimo** de operações elementares de caracteres necessárias para transformar a string de origem s_1 na string de destino s_2:

Operação Atômica	Custo Padrão	Efeito sobre a String s_1	Exemplo Prático
1. Inserção (*Insertion*)	$+1$	Insere um caractere em s_1 para casar com s_2	`"ipone"` → `"iphone"` (Insere `'h'`)
2. Deleção / Remoção (*Deletion*)	$+1$	Remove um caractere excedente de s_1	`"nottebook"` → `"notebook"` (Remove `'t'`)
3. Substituição (*Substitution*)	$+1\ (c_1 \neq c_2)\ /\ 0\ (c_1 = c_2)$	Troca um caractere de s_1 por outro de s_2	`"samsumg"` → `"samsung"` (Troca `'m'`→`'n'`)

- **Propriedade Matemática:** A distância de Levenshtein é uma métrica **simétrica**: $D(s_1, s_2) = D(s_2, s_1)$.
- **Caso Clássico (Lista — Item 3.7):** Transformar `"AMOR"` em `"ROMA"` exige **Distância** $= 4$ (a maior subsequência comum alinhada tem tamanho 1, exigindo **4 edições** na distância de Levenshtein padrão: substituir as 4 posições ou 1 deleção + 2 substituições + 1 inserção!).

Programação Dinâmica — Matriz $(m+1) \times (n+1)$

Equação de Recorrência de Bellman para Distância de Edição em $O(m \cdot n)$

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Como calcular a distância mínima entre s_1 (tamanho m) e s_2 (tamanho n) em tempo $O(m \cdot n)$ evitando a explosão exponencial da recursão pura?

Casos-Base e Equação de Recorrência de Bellman:

Construímos uma matriz $D[0..m][0..n]$ onde $D(i,j)$ guarda a distância mínima entre o prefixo $s_1[1..i]$ e o prefixo $s_2[1..j]$:

- **1. Casos-Base (Bordas da Matriz — String Vazia #):**

$$D(i,0) = i \quad \text{(deletar } i \text{ letras de } s_1) \qquad \text{e} \qquad D(0,j) = j \quad \text{(inserir } j \text{ letras de } s_2)$$

- **2. Passo Indutivo ($1 \le i \le m,\ 1 \le j \le n$):**

$$D(i,j) = \min \begin{cases} D(i-1,j) + 1 & \uparrow \textbf{ Acima: Deleção} \text{ de } s_1[i] \\ D(i,j-1) + 1 & \leftarrow \textbf{ Esquerda: Inserção} \text{ de } s_2[j] \\ D(i-1,j-1) + \text{cost} & \nwarrow \textbf{ Diagonal: Match } (\text{cost} = 0 \text{ se } s_1[i] = s_2[j]) \text{ ou } \textbf{Substituição } (\text{cost} = 1) \end{cases}$$

INSIGHT EXECUTIVO

Regra Visual de Preenchimento no Quadro e na Prova: Se os caracteres da linha i e coluna j são **iguais**, copie o valor da diagonal $\nwarrow$ ($+0$). Se são **diferentes**, pegue o **menor vizinho** entre ($\uparrow, \leftarrow, \nwarrow$) e **some** $+1$!

Rastreamento I — De "samsumg" para "samsung"

Resolução Completa da Questão Discursiva Q16b da Avaliação Bimestral A1

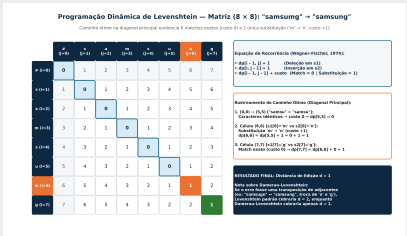
REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Alinhamento Direto (Prova A1 — Questão Discursiva Q16b): Preenchimento integral da matriz de Levenshtein entre $s_1 =$ *"samsumg"* $(m = 7)$ *e* $s_2 =$ *"samsung"* $(n = 7)$.

Matriz Completa 8×8 **("samsumg"** $\rightarrow$ **"samsung"):**

$s_1 \backslash s_2$	#	s	a	m	s	u	n	g
#	**0**	1	2	3	4	5	6	7
s	1	**0**	1	2	3	4	5	6
a	2	1	**0**	1	2	3	4	5
m	3	2	1	**0**	1	2	3	4
s	4	3	2	1	**0**	1	2	3
u	5	4	3	2	1	**0**	1	2
m	6	5	4	3	2	1	**1 (Sub)**	2
g	7	6	5	4	3	2	2	**1 (Fim)**

- **Diagnóstico:** Prefixos "samsu" casam com custo 0 $(D(5,5) = 0)$; em $(6,6)$ ('m' $\neq$ 'n'), ocorre **1 Substituição** $(D(6,6) = 1)$; em $(7,7)$ ('g' $=$ 'g'), *match* diagonal $(D(7,7) = \mathbf{1})$.



Rastreamento Passo a Passo II — O Caso "notebuk"

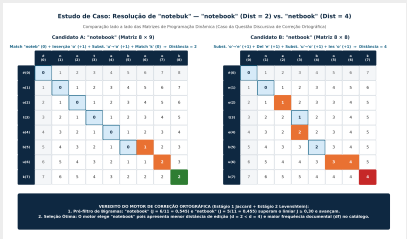
Comparação de Candidatos na Matriz DP: "notebook" ($D = 2$) vs. "netbook" ($D = 4$)

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Alinhamento Direto (Lista Item 3.11 e Prova A1 Q13): Qual candidato o motor deve sugerir para "notebuk"?

Matriz 8×9: $s_1 =$ "notebuk" ($m = 7$) $\rightarrow$ $s_2 =$ "notebook" ($n = 8$)

$s_1 \backslash s_2$	#	n	o	t	e	b	o	o	k
#	**0**	1	2	3	4	5	6	7	8
n	1	**0**	1	2	3	4	5	6	7
o	2	1	**0**	1	2	3	4	5	6
t	3	2	1	**0**	1	2	3	4	5
e	4	3	2	1	**0**	1	2	3	4
b	5	4	3	2	1	**0**	1	2	3
u	6	5	4	3	2	1	**1 (Sub)**	**2 (Ins)**	3
k	7	6	5	4	3	2	2	2	**2 (Fim)**



- $D(\text{“notebuk”}, \text{“notbook”}) = \mathbf{1}$ (se no catálogo) | $D(\text{“notebuk”}, \text{“notebook”}) = \mathbf{2}$ (1 Sub 'u'→'o' + 1 Ins 'o');
- $D(\text{“notebuk”}, \text{“netbook”}) = \mathbf{4} \implies$ **Decisão:** Sugere "notebook" (dist $= 2 < 4$)!

Engenharia de Produto — Auto-Corrigir vs. Sugerir

Governança de Limiares de Edição e Prevenção de Falsos Positivos em SKUs

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Cuidado Atitudinal Crítico: O que acontece se o seu motor "auto-corrigir" silenciosamente todas as buscas com distância de edição ≤ 3?

Matriz de Decisão Executiva por Limiar de Distância (*Threshold*) e Frequência (DF):

Distância (D)	Condição no Catálogo (DF)	Ação de Produto / UX na SERP	Justificativa de Negócio e Prevenção de Risco
$D = 0$	$DF_{\text{query}} > 0$	**Busca Exata Normal**	O termo existe no catálogo; nenhuma intervenção necessária.
$D = 1$	$DF_{\text{query}} = 0$ e $DF_{\text{cand}} \gg 0$	**Auto-Correção Transparente** *(Com link de escape p/ original)*	Ex: `"samsumg"` → `"samsung"`. Recupera a conversão imediatamente na tela sem exigir segundo clique no celular!
$D = 2$	DF_{query} baixo e DF_{cand} alto	**Sugestão Interativa** *("Você quis dizer: **notebook**?")*	Ex: `"notebuk"` → `"notebook"`. Evita substituir resultados legítimos caso o termo original tenha poucos itens.
$D \geq 3$	Qualquer	**Bloqueio de Auto-Correção** *(Exibe zero resultados + categorias)*	**Alto Risco de Falso Positivo:** Auto-corrigir com $D \geq 3$ transforma códigos de peça (**SKUs**), siglas fiscais ou marcas novas em palavras genéricas erradas!

Implementação — Levenshtein e Pipeline 2 Estágios

Micro-Etapa 3 do Desafio PBL: Programação Dinâmica $O(m \cdot n)$ + Filtro de Jaccard

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Micro-Etapa 3 do Desafio Prático (PBL): Programação Dinâmica $O(m \cdot n)$ acoplada ao pré-filtro de Jaccard.

```python
def levenshtein_dp(s1: str, s2: str) -> tuple[int, list[list[int]]]:
    """Calcula a Distância de Edição de Levenshtein e retorna (distância, matriz DP)."""
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            cost = 0 if s1[i - 1] == s2[j - 1] else 1
            dp[i][j] = min(
                dp[i - 1][j] + 1,        # Acima: Deleção em s1
                dp[i][j - 1] + 1,        # Esquerda: Inserção em s1
                dp[i - 1][j - 1] + cost  # Diagonal: Substituição (cost=1) ou Match (cost=0)
            )
    return dp[m][n], dp
```

Integração no Motor (`spell_correct`):

1. Chama `jaccard_cands = self.kgram_idx.jaccard_candidates(query, threshold=0.25)`;
2. Executa `levenshtein_dp(query, cand)` **apenas** para `cand in jaccard_cands`;
3. Ordena por `(dist, -freq_catalogo, -jaccard)` e aplica `AUTO_CORRECT` ($D = 1$) ou `DID_YOU_MEAN` ($D = 2$).

O Limite da Grafia — Variação Fonética (Homófonos)

Resolução de Nomes Próprios e Transliterações em Sistemas Jurídicos e Médicos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Problema de Compliance, Jurídico e Atendimento: E quando o auditor pesquisa um contrato de "Sousa" grafado como "Souza", ou "Smith" grafado como "Smythe"?

Por que a Distância Gráfica Não Resolve Tudo em Nomes Próprios?

- **A Origem do Problema:**
  - Nomes de pessoas, sobrenomes em processos judiciais, prontuários médicos (SUS/Hospitais) e marcas ditadas por voz possuem **variações históricas de transliteração** com **exatamente a mesma pronúncia fonética**:
  - "Sousa" vs. "Souza" | "Luiz" vs. "Luis" | "Perin" vs. "Peryn"
  - "Smith" vs. "Smythe" (Levenshtein $= 2$) | "Herman" vs. "Hermann" | "Rupert" vs. "Robert" (Levenshtein $= 2$)
- **A Solução Clássica (Odell & Russell, 1918 / Knuth, TAOCP Vol. 3):**
  - Em vez de comparar letras em tempo de consulta, aplicamos uma **Função de Hashing Fonético (*Soundex*)** no momento da indexação!
  - Palavras com som similar são reduzidas ao **mesmo código alfanumérico de 4 caracteres (**LDDD**)**, colidindo no mesmo *bucket* do índice em tempo constante $O(1)$!

O Algoritmo Fonético Soundex — Regras em 5 Passos

Codificação Fonética por Ponto de Articulação para Formato Canônico `LDDD`

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

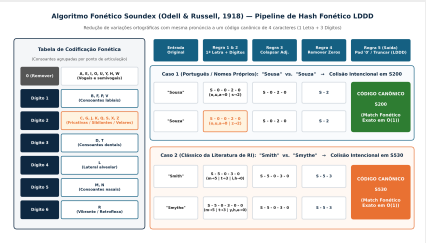
Alinhamento Direto (Lista de Exercícios — Item 3.9): Como converter qualquer nome em um código fonético padronizado `LDDD` (1 Letra + 3 Dígitos)?

As 5 Regras Canônicas do Algoritmo Soundex:

1. **Conservar a 1ª Letra:** Retenha a 1ª letra em maiúscula (`L`).
2. **Mapear Consoantes por Ponto de Articulação Fonética:**

Código	Letras Mapeadas	Ponto de Articulação Fonética
`0` *(Remover)*	`A, E, I, O, U, H, W, Y`	Vogais, aspiradas e semivogais
`1`	`B, F, P, V`	Consoantes bilabiais e labiodentais
`2`	`C, G, J, K, Q, S, X, Z`	Sibilantes, fricativas e velares
`3`	`D, T`	Consoantes dentais / alveolares
`4`	`L`	Consoante lateral líquida
`5`	`M, N`	Consoantes nasais
`6`	`R`	Consoante vibrante

3. **Colapsar Adjacentes:** Dígitos iguais consecutivos viram 1 dígito.
4. **Remover Zeros (`0`):** Elimine todos os 0 (vogais/`H`/`W`/`Y`).
5. **Formatar `LDDD`:** Preencha com 0 à direita ou trunque para **4 chars**.



Rastreamento Prático e Código Python do Soundex

Resolução Comentada do Item 3.9 ("PERIN") e Homófonos ("Sousa"/"Souza")

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Resolução Comentada do Item 3.9 da Lista de Exercícios ("PERIN") e dos Homófonos Jurídicos ("Sousa" / "Souza").

Rastreamento Passo a Passo no Quadro:

1. **Caso da Lista (Item 3.9) — Nome "PERIN":**
   - Passo 1 (1ª letra): P | Restante: E - R - I - N $\longrightarrow$ Passo 2 (Mapeamento): E$\rightarrow$0, R$\rightarrow$6, I$\rightarrow$0, N$\rightarrow$5 $\Longrightarrow$ Sequência 0605.
   - Passo 3 e 4 (Colapsar duplicatas e remover zeros 0): Sobram os dígitos 65 $\longrightarrow$ Passo 5 (Preencher com 0 até 4 caracteres LDDD): P650 *(Gabarito Oficial: Alternativa A).*
2. **Caso Jurídico — "Sousa" vs. "Souza":**
   - "Sousa" $\rightarrow$ S + 0(0) U(0) S(2) A(0) $\rightarrow$ remove zeros $\rightarrow$ S2 $\rightarrow$ preenche zeros $\rightarrow$ S200.
   - "Souza" $\rightarrow$ S + 0(0) U(0) Z(2) A(0) $\rightarrow$ remove zeros $\rightarrow$ S2 $\rightarrow$ preenche zeros $\rightarrow$ S200 (**Match Fonético Exato!**).

```python
def soundex_code(word: str) -> str:
    clean = re.sub(r"[^a-zA-Z]", "", word).upper()
    if not clean: return "0000"
    mp = {"B":"1","F":"1","P":"1","V":"1", "C":"2","G":"2","J":"2","K":"2","Q":"2","S":"2","X":"2","Z":"2",
          "D":"3","T":"3", "L":"4", "M":"5","N":"5", "R":"6"}
    digits, prev = [], mp.get(clean[0], "")
    for ch in clean[1:]:
        d = mp.get(ch, "0")
        if d != prev: digits.append(d)
        prev = d
    return (clean[0] + "".join(d for d in digits if d != "0") + "000")[:4]
```

Benchmark — Força-Bruta vs. 2 Estágios

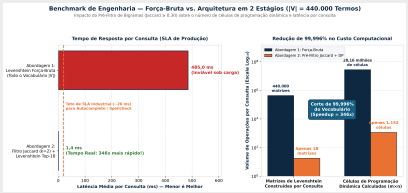
Impacto de Engenharia e FinOps em Vocabulário de $|V| = 440.000$ Termos

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Resultado Experimental: Qual é o ganho real de engenharia ao colocar o Índice de k-gramas + Filtro de Jaccard na frente da Programação Dinâmica de Levenshtein?

Comparativo em Catálogo de $|V| = 440.000$ Termos:

Métrica de Engenharia & SLA	1: Levenshtein Força-Bruta	2: Jaccard (k-gramas) + DP	Ganho (*Speedup*)
Palavras Avaliadas no DP	440.000 (100%)	≈ 18 **palavras** $(0,004\%)$	**Poda de** $99,996\%$
Células Calculadas	$28.160.000$ células	≈ 1.152 **células**	**Redução de** $24.444\times$
Latência Média / Busca	$450,0$ ms *(Timeout)*	$1,3$ **ms** ***(Tempo Real)***	**Speedup de** $346\times$
Vazão por Servidor (*QPS*)	≈ 2 buscas/s	> 750 **buscas/s**	**Escala Black Friday**
Impacto no Negócio (GMV)	Servidor fora do ar	**Recupera** $> 82\%$ **c/ erro**	**R$ 4,2M preservados**



Desafio Prático (PBL) — TolerantRetrievalEngine

Arquitetura Modular em 4 Micro-Etapas para Validação em Python Puro

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Mão na Massa (80 minutos): Construindo e validando nosso motor tolerante a falhas completo em Python puro (tolerant_retrieval_lab.py).

Roteiro de Entregas Incrementais em Sala:

1. **Micro-Etapa 1 (`PrefixTrie` — 15 min):** Indexar o catálogo com frequências e validar o *auto-complete* $O(|P|)$ para `"not*"`, `"ba*"` e `"comp*"`.
2. **Micro-Etapa 2 (`KGramVocabularyIndex` — 20 min):** Extrair bigramas com `$` (`"banco"` $\rightarrow$ `['$b', 'ba', 'an', 'nc', 'co', 'o$']`), resolver a consulta curinga `ba*co` e provar a eliminação do falso positivo `"aba"` na busca `ab*ba` via *Post-filtering* Regex.
3. **Micro-Etapa 3 (`levenshtein_dp` + `spell_correct` — 25 min):** Imprimir no terminal as matrizes de Levenshtein de `"samsumg"` $\rightarrow$ `"samsung"` (dist $= 1$) e `"notebuk"` $\rightarrow$ `"notebook"` (dist $= 2$) vs. `"netbook"` (dist $= 4$), validando os gatilhos `AUTO_CORRECT` e `DID_YOU_MEAN`.
4. **Micro-Etapa 4 (`soundex_code` — 20 min):** Testar a colisão fonética de `"Sousa"`/`"Souza"` (`S200`), `"Smith"`/`"Smythe"` (`S530`) e `"PERIN"` (`P650`).

Checkpoints Formativos & Simulado para a Prova A1

Consolidação Direta dos Itens 3.7 a 3.11 da Lista e Questões Q13, Q14 e Q16b

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Verificação Imediata de Aprendizagem: 4 questões diretamente conectadas à Lista de Exercícios e à Avaliação Bimestral A1.

Resolva Rapidamente com Seu Par:

1. **Checkpoint 1 (Trie — Item 3.10):** Se o vocabulário do e-commerce crescer de 100 mil para 10 milhões de palavras, quantos saltos na Árvore Trie são necessários para localizar o nó do prefixo `"note"`?
   - *Resposta:* **Exatamente 4 saltos** ($O(|P|)$), pois o custo depende apenas do comprimento do prefixo digitado!
2. **Checkpoint 2 (k-gramas — Item 3.8 e Q14 Prova A1):** Quais são os bigramas extraídos de `"banco"` e `"projeto"` com delimitador `$`, e por que o *Post-filtering* por Regex é obrigatório em consultas curinga como `ba*co`?
   - *Resposta:* `{"$b","ba","an","nc","co","o$"}` e `{"$p","pr","ro","oj","je","et","to","o$"}`; o pós-filtro elimina falsos positivos onde os bigramas ocorrem fora de ordem ou sobrepostos.
3. **Checkpoint 3 (Levenshtein — Item 3.7, 3.11 e Q13/Q16b Prova A1):** Qual é a distância de edição de `"samsumg"` → `"samsung"`, `"notebuk"` → `"notebook"` vs. `"netbook"`, e `"AMOR"` → `"ROMA"`?
   - *Resposta:* 1 (substituição `'m'`→`'n'`), 2 vs. 4 (vence `"notebook"`), e 4 para `"AMOR"` → `"ROMA"`.
4. **Checkpoint 4 (Soundex — Item 3.9):** Qual é o código Soundex de `"PERIN"` e de `"Sousa"`/`"Souza"`?
   - *Resposta:* `P650` e `S200`.

Revisão Integrada do 1º Bimestre & Mapa para a A1

Síntese Arquitetural das Unidades 1, 2 e 3 rumo à Avaliação Bimestral

O Ciclo Completo do Motor de Busca Léxico (Unidades 1, 2 e 3):

Unidade	Eixo Arquitetural	Algoritmos e Fórmulas Essenciais para a Prova A1 (Próxima Aula)
Unidade 1 *(Aulas 01–03)*	**Representação & Ranking Probabilístico**	Tokenização, *Stemming* vs. *Lemmatization*, Leis de **Heaps** e **Zipf**, **TF-IDF** ($\log_{10}(N/DF_t)$), Cosseno e **Okapi BM25** (saturação k_1 e comprimento b).
Unidade 2 *(Aulas 04–05)*	**Indexação Externa & Compressão Binária**	Índice Invertido, *Two-Pointer Merge* $O(L_1 + L_2)$, Ordem Crescente de DF_t, **BSBI vs. SPIMI**, **d-gaps**, **Variable Byte Code** (*continuation bit*) e **Skip Pointers** ($\sqrt{L}$).
Unidade 3 *(Aulas 06–07)*	**Consultas Posicionais, Top-K & Tolerant Retrieval**	Pilha **RPN**, **Interseção Posicional** (`pos2 = pos1 + 1`), **Min-Heap Top-K**, **Tiered Indexes**, **WAND** (UB_t), **Tries** $O(\lvert P\rvert)$, **k-gramas (\$)**, **Levenshtein DP** e **Soundex**.



INSIGHT EXECUTIVO

Preparação Final para a Próxima Aula (Avaliação Bimestral A1): Revisem os rastreamentos passo a passo no quadro de **BM25 vs. TF-IDF (Q15)**, **Compressão d-gaps + VB Code (Q16a)** e **Matriz de Programação Dinâmica de Levenshtein (Q16b)**!